

Reviewing fundamental concepts in Python

function, methods, files, list, loops

CS 22B, Spring 2026

Module 01: Review

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA



Define function and
method



Working with files



List objects

Python for Biologists



A programming course for complete beginners

by Dr. Martin Jones

Python Review

We will review the topics covered in
textbook

<http://userpages.fu-berlin.de/digga/p4b.pdf>

by Dr. Martin Jones

Run Python programs

Execute Python code on a command line

- Linux or Mac: terminal
- Windows: command prompt

eg., Mac

```
% python3
Python 3.13.0 (main, Oct  7 2024, 05:02:14) [Clang 15.0.0
(clang-1500.1.0.2.5)] on darwin
Type "help", "copyright", "credits" or "license" for more
information.
```

Run Python programs

05

```
% python3
```

```
Python 3.11.5 (main, Sep 11 2023, 08:19:27) [Clang  
14.0.6 ] on darwin  
Type "help", "copyright", "credits" or "license" for  
more information.  
>>> print("Hello")  
Hello
```

```
>>> print("Hello")
```

function

argument

statement

```
>>> my_string = "Data is fun"  
>>> my_int = 4
```

You can store string, int, or float in a variable

Tip:

- Use meaningful name for your variable name
- Remember names are case-sensitive

Built-in data types

numbers

```
>>> my_string = "Data is fun"  
>>> length_str = len(my_string)
```

- `my_string` is a *str* datatype
- `length_str` is a *int* datatype

```
>>> print(my_string + " has a length of " +  
str(length_str))
```

Other built-in *number data types* include floats, complex numbers, booleans.

Number data types can be manipulated using arithmetic operators

Built-in data types

lists, tuples, string, set, file objects

lists – [1, "two", 3.0, 4, ["a", "b"]]

Can contain mixture of data types, mutable, and is indexed.

tuples – (1, "two", 3.0, 4, ["a", "b"], (5,6))

Can contain mixture of data types, immutable, and is indexed.

strings – "A string"

dictionaries – x = {1:"one", 2:"two", 3:"three"}

key:value pairs where value is mutable

set – x = set([1, 2, 3, 4, 3, 2])

Unorder collection of objects that are unique


```
>>> my_string = "Data is fun"  
>>> print(my_string.lower())  
data is fun
```

```
>>> print(my_string.lower())
```

variable

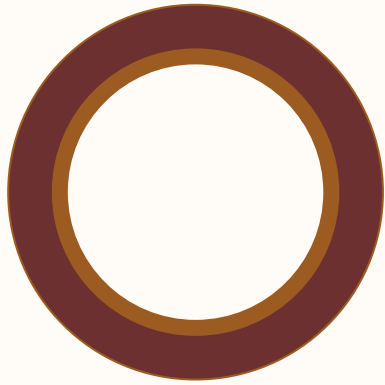
method

argument

Method – similar to a function, but is a set of instructions associated with an object.

***In example, the method is associated with `string` type*

Method



Chapter 2: Focus on working and manipulating text



Printing and Manipulating Text

11

```
>>>my_dna = "ATGCGTA"————— Assign string to a variable
```

Different methods you can do with **str** type

- `my_dna.lower()`
- `my_dna.replace("A", "T")`
- `my_dna.count("A")`
- `my_dna[2:5] #start: stop -1 :skip`



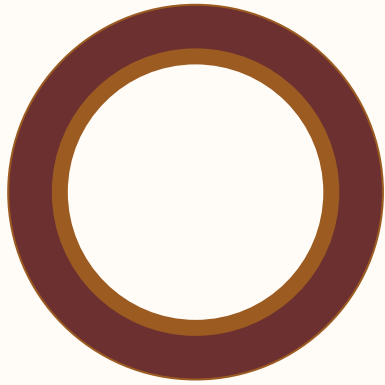
Class exercise

12

Given sequence "ATTGGCTATACCGG"

CL1.1: Count the number of T in the sequence

CL1.2: For the sequence above, convert the sequence from the 4th character to the 9th character to lowercase



Chapter 3: Focus on working with files (input/output)



Reading and Writing Files

14

Reading text from a file

- A text file does not need to be human readable.
- *Question:*

What are examples of non-human readable files?



Reading and Writing Files

Opening a file using function **open()**

open() takes in an *argument* and returns a *file object*



```
>>>file_obj = open("my_file.txt")
```

└──┴──┬──────────┴──┐
method argument/file name

Read the contents of the file using *read* method

```
>>>>file_content = my_file.read()
```

Working with file

16

Opening, reading, writing and appending a txt file

```
>>>file_obj = open("seq.txt")
```

Different **modes** you can do with **file**

- `file_obj.read()`
- `file_obj.write()`
- `file_obj.append()`
- `file_obj.close()`

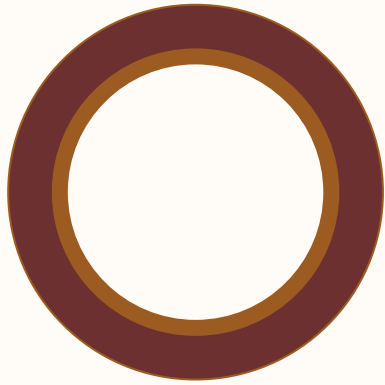
Class exercise

16

Given the file “seq.txt”

CL1.3: Find the coding and non-coding regions

1. Read in the seq.txt file
2. Write two new files; coding.txt and noncoding.txt.
3. Concatenate the exons (uppercase) to the coding file and intron (lowercase) to the noncoding file
4. Close coding and noncoding regions



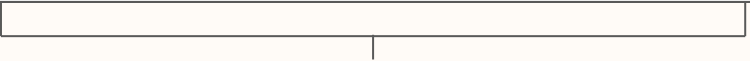
Chapter 4: Working with multiple elements and repetition



List – data type structure that can store multiple pieces of information at the same time

* note: there are 4 built-in data type used to store collection of data; tuple, set, and dictionary

```
>>> fruit = ["apple", "lemon", "banana"]  
  
>>> fruit  
  
['apple', 'lemon', 'banana']
```



elements

Retrieving element from list

```
>>> first_fruit = fruit[0]
```

Use index method to retrieve element when you do not know index

```
>>> where_banana = fruit.index("banana")
```

To get last element

```
>>> last_fruit = fruit[-1]
```

```
>>> fruit = list(("apple", "banana", "oranges", "lime"))
```

```
fruit.append("kiwi")
```

```
fruit.reverse()
```

```
fruit.sort()
```

Working with list elements

Loops

22

Want output

apple is a fruit

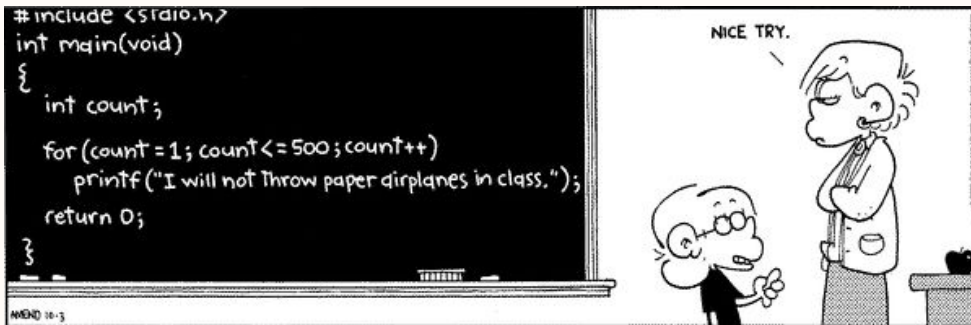
banana is a fruit

orange is a fruit

lime is a fruit

Want "for each element in the list of fruit, print out the element, followed by the words ' is a fruit."

```
for f in fruit:  
    print(f + " is a fruit.")
```



Class exercise

23

CL1.4:

1. File "adapter_input.txt" contains a number of DNA sequence, one per line. Each sequence starts with a sequencing adapter comprise of 14 bp (14 characters), "ATTCGATTATAAGC".
2. Write a program that will trim/remove the adapter sequence.
3. Write the edited/clean sequence to a new file, "clean_input.txt".
4. Additionally, print out the length of each sequence to the screen.

FIN

Next class

.